

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

GUÍA DE PRÁCTICA EXPERIMENTAL

1 DATOS GENERALES

CARRERA: SOFTWARE (REDISEÑO)

ASIGNATURA: APLICACIONES WEB [111]

NIVEL / PARALELO: 5TO NIVEL – A

N.º DE ESTUDIANTES:

PROFESOR: GUERRERO ULLOA GLEISTON CICERON

PERIODO ACADÉMICO: 2026-2027 PPA

FECHA: 01-05-2026

TIPO DE GUÍA: PRÁCTICA EXPERIMENTAL

TIPO DE UBICACIÓN: INTERNA

TIPO AULA/LABORATORIO: LABORATORIOS

AULA/LABORATORIO: FCCDD-LAB-TIC-202

UBICACIÓN: CAMPUS CENTRAL

TIPO DE APRENDIZAJE: Resolución de problemas prácticos

N. HORAS: 12

2 DATOS ACADÉMICOS

Resultado de aprendizaje

Diseña e implementa soluciones web que integran mecanismos de gestión de estado, acceso a bases de datos relacionales mediante SQL parametrizado y ORM, y patrones de arquitectura escalables (multicapa, SOA, EDA, P2P), documentando las decisiones arquitectónicas con herramientas estándar.

Tema de la práctica

Acceso a Datos y Patrones de Arquitectura: Integración ORM, Caché con Redis y Diseño Arquitectónico C4

3 OBJETIVO

Objetivo General

Diseñar e implementar la capa de integración del Proyecto Fin de Curso, conectando el frontend con el backend a través de una arquitectura documentada con Modelo C4, implementando acceso a datos con ORM y patrón Repository, caché con Redis (patrón cache-aside), y midiendo el impacto cuantitativo de la caché en el rendimiento de las consultas, con cobertura de pruebas unitarias $\geq 70\%$.

Objetivos Específicos:

- OE1. **Documentar** la arquitectura completa del PFC con el Modelo C4 en los niveles de Contexto, Contenedores y Componentes, incluyendo mínimo 3 ADR que justifiquen las decisiones arquitectónicas más relevantes.
- OE2. **Implementar** el modelo de datos completo del PFC con ORM (Doctrine, Entity Framework Core o Hibernate/JPA), incluyendo migraciones versionadas, seeders con ≥ 50 registros realistas, y CRUD con paginación, ordenamiento dinámico y búsqueda con filtros múltiples.
- OE3. **Implementar** el patrón de caché cache-aside con Redis, midiendo el speedup de la caché en la consulta principal del PFC con la fórmula $S = T_{sin}/T_{con}$ en 10 repeticiones.
- OE4. **Desarrollar** pruebas unitarias para la capa Repository con cobertura $\geq 70\%$, generando el reporte de cobertura e incluyéndolo en el informe.

4 FUNDAMENTO TEÓRICO

Importante:

Desarrollar el fundamento teórico completo de la Unidad III. Incluir diagramas propios (C4, secuencia, E-R), no solo descripciones textuales. El análisis cuantitativo de rendimiento es obligatorio.

5.1 Gestión de Estado en Aplicaciones Web: Request, Response, Session

El equipo debe desarrollar (mínimo 350 palabras):

- **La naturaleza stateless de HTTP y sus implicaciones:** HTTP es un protocolo sin estado por diseño (RFC 7230). Explicar por qué esto es una ventaja para la escalabilidad (cada servidor puede responder cualquier solicitud) y un desafío para las aplicaciones con estado (carrito de compras, sesiones de usuario). Describir los mecanismos de estado: cookies, sesiones del servidor, tokens JWT (stateless), y bases de datos. Comparativa de mecanismos de almacenamiento client-side: tabla técnica de localStorage vs. sessionStorage vs. cookies: capacidad, persistencia, accesibilidad desde JavaScript, envío automático con HTTP, riesgos XSS (localStorage accesible desde JS, cookie HttpOnly no), riesgos CSRF (solo cookies se envían automáticamente). Recomendar cuál usar en cada escenario del PFC. Sesiones escalables con Redis: describir el problema del sticky session (todas las peticiones de un usuario van al mismo servidor, limita el balanceo de carga). Solución: centralizar las sesiones en Redis: todos los servidores acceden al mismo almacén. Comparar el almacenamiento de sesiones en: (a) sistema de archivos, (b) base de datos relacional, (c) Redis — velocidad, persistencia, atomicidad.
- **Objetos Request y Response en el framework del PFC:** describir con código cómo se accede a los datos de la petición en el framework elegido (Laravel, ASP.NET Core o Spring Boot). Mostrar el ciclo completo: desde la llegada de la petición HTTP hasta la respuesta JSON o HTML.

Referencias mínimas IEEE:

- R. Nixon, Learning PHP, MySQL & JavaScript, 6th ed. O'Reilly Media, 2021, ISBN: 978-1-492-06229-9.
- L. Welling and L. Thomson, PHP and MySQL Web Development, 5th ed. Addison Wesley, 2016, ISBN: 978-0-321-83389-1.

5.2 ORM y Mapeo Objeto-Relacional

El equipo debe desarrollar (mínimo 350 palabras):

- **El problema del impedance mismatch:** explicar formalmente por qué el modelo relacional (tablas, filas, joins) y el modelo orientado a objetos (clases, herencia, asociaciones) son paradigmas fundamentalmente diferentes. Describir las estrategias de mapeo de herencia en ORM: single table inheritance, class table inheritance, concrete table inheritance — cuándo usar cada una.
- **Comparativa ORM vs. SQL puro vs. query builder:** construir una tabla con criterios: curva de aprendizaje, rendimiento en consultas simples, rendimiento en consultas complejas (N+1 problem), portabilidad entre SGBD, facilidad de migración del esquema, debugging/logging de queries generadas.
- **El problema N+1 y lazy/eager loading:** describir el problema N+1 con un ejemplo concreto del PFC (p.ej. listar usuarios con sus pedidos). Mostrar cómo se produce en Eloquent (sin with()) y cómo se soluciona con eager loading (User::with('pedidos')->get()). Medir la diferencia en número de queries con el log de queries de Laravel/Doctrine.
- **Migraciones y gestión del esquema:** comparar las estrategias de migración: migraciones versionadas (Laravel, EF Core, Flyway), schema generation automática (Hibernate hbm2ddl.auto=update) y herramientas dedicadas (Liquibase). Analizar los riesgos de cada enfoque en producción.

Referencias mínimas IEEE:

- M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002, ISBN: 978-0-321-12521-7.
- A. Lock, ASP.NET Core in Action, 2nd ed. Manning Publications, 2021, ISBN: 978-1-61729-678-7.

5.3 Patrones de Arquitectura para Aplicaciones Web Escalables

El equipo debe desarrollar (mínimo 350 palabras):

- **Modelo C4 para documentar arquitectura:** explicar los 4 niveles del Modelo C4 de Simon Brown: (C1) Contexto — el sistema y sus usuarios/sistemas externos; (C2) Contenedores — aplicaciones y bases de datos que componen el sistema; (C3) Componentes — módulos dentro de cada contenedor; (C4) Código — clases (solo para partes críticas). ¿Por qué C4 es superior a UML de alto nivel para comunicar arquitectura a diferentes audiencias?
- **Arquitectura por capas vs. microservicios — análisis para el PFC:** comparar arquitectura monolítica por capas (presentación, negocio, datos) vs. microservicios: acoplamiento, despliegue, escalado independiente, complejidad operacional. Para un PFC universitario con equipo de 3 personas, ¿cuál es la arquitectura más adecuada y por qué? Argumentar con criterios de team cognitive load y distributed systems fallacies.
- **Event-Driven Architecture (EDA) y casos de uso web:** describir los componentes de EDA: productores de eventos, event bus (RabbitMQ, Apache Kafka), consumidores. WebSockets como caso particular de EDA para comunicación en tiempo real: el protocolo upgrade (HTTP → WebSocket), el modelo de eventos bidireccional. ¿Cuándo justifica el PFC usar EDA vs. REST síncrono?

- **ADR como práctica de gobernanza arquitectónica:** justificar por qué documentar las decisiones arquitectónicas (no solo el resultado) es parte del proceso de ingeniería de software según SWEBOK v4.0. Describir el formato estándar del ADR propuesto por Michael Nygard y su adopción en equipos modernos.

Referencias mínimas IEEE:

- L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley, 2021, ISBN: 978-0-13-688080-1.
- M. T. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018, ISBN: 978-1-68050-239-8.

5.4 Caché: Patrones y Medición de Rendimiento

El equipo debe desarrollar (mínimo 350 palabras):

- **Niveles de caché en una aplicación web:** describir los niveles: caché del navegador (Cache-Control, ETag, Last-Modified), CDN (distribución geográfica de contenido estático), caché de aplicación (Redis/Memcached), caché de base de datos (query cache, buffer pool de MySQL). Para cada nivel: qué tipo de datos cacheará, TTL apropiado, estrategia de invalidación.
- **Patrones de caché: cache-aside, read-through, write-through, write behind:** para cada patrón: describir el flujo de datos con diagrama de secuencia, cuándo usar, ventajas y desventajas. El patrón cache-aside es el más común en aplicaciones web: la aplicación primero busca en caché, si no está (cache miss) consulta la BD y almacena en caché. El cache stampede es un problema cuando muchas peticiones experimentan un miss simultáneo: describir la solución con mutex o probabilistic early expiration.
- **Redis: estructura de datos y comandos fundamentales:** Redis no es solo un caché de clave-valor simple; soporta strings, hashes, lists, sets, sorted sets y streams. Para el PFC: usar strings para objetos JSON serializados (SET clave valor EX 300), hashes para entidades (HSET usuario:1 nombre .Ana"), sorted sets para rankings. Mostrar la integración con Predis (PHP) o StackExchange.Redis (.NET).
- **Metodología de benchmarking de rendimiento web:** describir cómo medir el impacto de la caché de manera estadísticamente válida: $n \geq 10$ repeticiones para calcular media y desviación estándar, verificar que la diferencia es estadísticamente significativa, aislar la variable (misma query, mismas condiciones de red). Mostrar cómo usar el log de queries del ORM y microtime(true) en PHP.

Referencias mínimas IEEE:

- M. T. Nygard, Release It!, 2nd ed. Pragmatic Bookshelf, 2018, ISBN: 978-1-68050 239-8.
- M. Fowler, Patterns of Enterprise Application Architecture. Addison-Wesley, 2002, ISBN: 978-0-321-12521-7.

5 MATERIALES, EQUIPOS, INSUMOS Y REACTIVOS

Materiales

- Docker Desktop 4.x con Docker Compose 2.x
- Redis 7.x (imagen Docker redis:7-alpine)
- MySQL 8.0 (imagen Docker mysql:8.0)
- Predis (PHP): <https://github.com/predis/predis>
- StackExchange.Redis (.NET) o Jedis (Java): según tecnología elegida
- Xdebug 3.x (para cobertura PHP): <https://xdebug.org>
- PHPUnit 11.x: <https://phpunit.de>

Equipos

- Procesador: Intel Core i5 o AMD Ryzen 5 RAM 16 GB HD 15 GB libres Docker y BD

Insumos y reactivos

- Redis Commander(GUI de Redis, Docker): <https://github.com/joeferner/redis-commander>
- draw.io: diagramas C4 con plantillas, gratuito
- dbdiagram.io: diagramas E-R online, gratuito
- Structurizr: <https://structurizr.com/dsl> (C4 gratuito)

6 PROCEDIMIENTOS

Paso 1– Diseño arquitectónico con Modelo C4

1.1 Diagrama C4 Nivel 1: Contexto del sistema

El diagrama de contexto muestra el PFC como una caja negra e identifica los actores y sistemas externos que interactúan con él.
Herramientas:

- Structurizr Lite (gratuito): <https://structurizr.com/dsl> — código DSL
- C4-PlantUML: <https://github.com/plantuml-stdlib/C4-PlantUML>
- draw.io con plantillas C4: <https://app.diagrams.net>

Elementos mínimos del diagrama C4 Nivel 1:

- El sistema (caja central)
- Usuarios que interactúan (al menos: usuario final, administrador)
- Sistemas externos (BD, API externa, servicio de correo si aplica)
- Relaciones etiquetadas con el protocolo usado

1.2 Diagrama C4 Nivel 2: Contenedores

Mostrar los contenedores de software: aplicación web (frontend), servidor de aplicaciones (backend), base de datos relacional, Redis (caché), y si aplica: CDN, servicio de correo. Para cada contenedor: tecnología usada y responsabilidad principal.

1.3 Diagrama C4 Nivel 3: Componentes

Para el contenedor de mayor complejidad (backend), descomponer en componentes: Controladores, Servicios, Repositorios, Middleware de autenticación, ORM/Entity Layer. Mostrar las dependencias entre componentes.

1.4 Documentar 3 ADR

Crear en docs/adr/:

- ADR-001: Selección de patrón de arquitectura (monolítica por capas justifi cada)
- ADR-002: Selección del ORM (Doctrine vs. EF Core vs. Hibernate)
- ADR-003: Estrategia de caché (Redis con patrón cache-aside)

Criterio de verificación: Los 3 diagramas C4 están en formato PNG en el repositorio (docs/arquitectura/). Los 3 ADR están completos con todas las secciones del formato estándar.

Paso 2– ORM, migraciones y CRUD avanzado

2.1 Configuración del ORM con Docker Compose actualizado.

2.2 Migraciones con el ORM elegido

2.3 CRUD con paginación y filtros (Laravel ejemplo)

Criterio de verificación: Las migraciones corren sin errores. El seeder crea ≥50 registros. El endpoint de listado devuelve datos paginados con metadata de paginación (current_page, total, last_page).

Paso 3– Caché con Redis: patrón cache-aside y benchmarking

3.1 Implementar patrón cache-aside

3.2 Benchmark: medir el speedup de la caché

Registrar los resultados en la siguiente tabla (incluir en el informe):

Iteración	Sin caché (ms)	Con caché (ms)
1		
2		
...		
10		
Media		
Speedup	$S = T_{\text{sin}}/T_{\text{con}} = ____ \times$	
S		

Criterio de verificación: El speedup $S > 1$ (la caché es más rápida). Documentar en el informe si el speedup fue estadísticamente significativo ($S > 2$ es un umbral típico para justificar la complejidad de Redis).

Paso 4– Pruebas unitarias del Repository

4.1 Configurar PHPUnit con base de datos en memoria (SQLite)

Criterio de verificación: El reporte de cobertura muestra $\geq 70\%$ para los métodos de la clase Repository. Captura incluida en el informe.

Paso 5— Integración y análisis de escalabilidad

5.1 Integración frontend-backend-datos

Verificar que el flujo completo del PFC funciona:

1. El usuario accede al frontend (PE-U1) desde `http://localhost:5500`
2. El frontend hace una petición `fetch()` al backend (PE-U2) en `http://localhost:8080/api/entidades`
3. El backend consulta el Service, que busca en Redis (cache-aside)
4. Si cache miss: el Service consulta el Repository, que usa el ORM para consultar MySQL
5. Los datos regresan al frontend y se renderizan en tarjetas
6. Capturar el flujo completo en el Network tab de DevTools

5.2 Análisis de escalabilidad horizontal

Describir en el informe (con diagrama): si el PFC tuviera 10.000 usuarios concurrentes, ¿cómo escalaría horizontalmente? Considerar:

- Balanceador de carga (Nginx como reverse proxy)
- Múltiples instancias del backend (Docker Compose replicas: 3)
- Sesiones centralizadas en Redis (sin sticky sessions)
- Base de datos con réplica de lectura (read replica)

Criterio de verificación: El flujo completo frontend-backend-datos funciona sin errores. La prueba de caché tiene resultados numéricos documentados.

7 BIBLIOGRAFÍA

- L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley, 2021, ISBN: 978-0-13-688080-1.
- M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002, ISBN: 978-0-321-12521-7.
- M. T. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018, ISBN: 978-1-68050-239-8.
- R. Nixon, Learning PHP, MySQL & JavaScript, 6th ed. O'Reilly Media, 2021, ISBN: 978-1-492-06229-9.
- H. Washizaki, Ed., Guide to the Software Engineering Body of Knowledge (SWE BOK) Version 4.0. Piscataway, NJ, USA: IEEE Computer Society, 2024.
- Laravel, "Laravel 11.x Documentation," 2024. [Online]. Available: <https://laravel.com/docs/11.x>
- PHPUnit, "PHPUnit 11 Manual," 2024. [Online]. Available: <https://phpunit.de/documentation.html>

8 ANEXOS

- [Análisis]: En el benchmark de caché, el speedup obtenido fue $S = ___x$. ¿Este resultado justifica la complejidad operacional de añadir Redis al stack del PFC? Considere: costos de infraestructura adicional, complejidad de despliegue, posibles fallos de Redis y su impacto en la disponibilidad.
- [Síntesis]: Diseña la estrategia de invalidación de caché para el módulo principal del PFC: ¿qué claves de Redis se invalidan cuando se crea, actualiza o elimina una entidad? ¿Hay riesgo de cache stampede en el PFC? ¿Cómo lo mitigaría?
- [Evaluación]: Compare el ADR-001 (decisión de arquitectura) del equipo con la arquitectura de al menos un sistema web de producción documentado en la literatura. ¿Qué aspectos del PFC podrían requerir una arquitectura diferente a medida que escala?
- [Evaluación]: La cobertura de pruebas del Repository alcanzó $___%$. ¿Qué casos de prueba adicionales aumentarían la confianza en el código sin aumentar significativamente el mantenimiento de los tests? ¿Qué tipo de bugs no captura la prueba unitaria del Repository?
- Anexo A— Estructura del repositorio (rama feature/data-layer) Se muestra en el archivo adjunto
- Anexo C— Preguntas de Análisis

9 RESULTADOS DE ESTUDIANTES

EQUIPO A

ZAMBRANO MOREIRA ALISON ARIANA
UMAGINGA AREVALO JEFFERSON MANUEL
MONCAYO LOOR XAVIER ALEJANDRO

Resultados obtenidos

El desarrollo del **Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR)** permitió diseñar una arquitectura moderna basada en **Spring Boot, Angular, PostgreSQL, JWT y Redis**, garantizando una separación adecuada entre las capas de presentación, lógica de negocio y acceso a datos. Se elaboraron los principales modelos de diseño y documentación técnica, entre ellos los diagramas **C4, casos de uso, clases, secuencia y modelo entidad-relación**, los cuales describen el funcionamiento interno del sistema y facilitan su mantenimiento.

En el ámbito de la seguridad, se implementó un mecanismo de autenticación mediante **JSON Web Tokens (JWT)** almacenados en **cookies HttpOnly**, reduciendo el riesgo de ataques relacionados con el acceso al token desde el cliente. Además, el uso de **Redis** permitió gestionar el cierre de sesión y la revocación de tokens mediante una lista negra temporal, incrementando la seguridad del sistema. Finalmente, la documentación desarrollada proporciona una base sólida para futuras mejoras e implementación del sistema en un entorno real.

Conclusión

- Se diseñó un sistema de gestión bibliotecaria con una arquitectura escalable y organizada, aplicando buenas prácticas de desarrollo y documentación mediante diagramas UML y el modelo C4.
- La implementación de autenticación basada en **JWT**, junto con **cookies HttpOnly** y **Redis** para la revocación de tokens, permitió fortalecer la seguridad del sistema y mejorar el control sobre las sesiones de los usuarios.
- La integración de tecnologías como **Spring Boot, Angular** y **PostgreSQL** demuestra que es posible desarrollar una solución moderna, segura y preparada para futuras ampliaciones, facilitando la administración de préstamos, usuarios y libros mediante el uso de códigos QR.

EQUIPO B

TAIPE MORA ZAIDA MELISSA
PANAMA MURILLO MOISES ANTONIO
CAJAS IBARRA IRVIN MARCELO

Resultados obtenidos

La práctica permitió implementar satisfactoriamente las funcionalidades requeridas en el proyecto. Se integró autenticación mediante JWT con almacenamiento en cookies HttpOnly y blacklist de JTI en Redis, verificando la invalidez del token tras el cierre de sesión. Se desarrolló un CRUD completo utilizando Spring Data JPA con paginación mediante Pageable y una base de datos inicial con más de 50 registros cargados mediante Flyway. Además, se implementaron la arquitectura en capas y el patrón cache-aside con Redis para optimizar consultas. Se elaboraron los diagramas UML, ER, C4 y los ADR solicitados. Finalmente, las pruebas comparativas evidenciaron una reducción significativa en los tiempos de respuesta gracias al uso del sistema de caché.

Conclusión

La práctica experimental permitió consolidar conocimientos sobre el desarrollo de aplicaciones empresariales modernas empleando Java 21, Spring Boot, Angular, PostgreSQL y Redis. La implementación de mecanismos de seguridad, persistencia, arquitectura y almacenamiento en caché fortaleció la comprensión de buenas prácticas de diseño y desarrollo. Asimismo, la documentación mediante diagramas y ADR facilitó la justificación de las decisiones arquitectónicas adoptadas. Los resultados obtenidos demostraron que la incorporación de Redis mejora el rendimiento del sistema y que la arquitectura implementada favorece la escalabilidad, el mantenimiento y la seguridad de la aplicación.

EQUIPO C

ZAMORA BUMBILA DIEGO ALEXANDER
ZAMORA ARIAS CARLA ESTHEFANIA
ARCALLE GREFA DARWIN ORLANDO

Resultados obtenidos

Se logró implementar un backend robusto y seguro para el SGED. La autenticación *stateless* mediante JWT en cookies *HttpOnly*, complementada con la lista negra de JTIs en Redis, garantizó un proceso de cierre de sesión inmediato e inviolable que responde en menos de 2 ms. Por otro lado, el CRUD con Spring Data JPA y Flyway cargó con éxito 50 registros semilla, asegurando listados fluidos gracias a la paginación con Pageable. Finalmente, la adopción de la arquitectura en capas estructuró el código limpiamente (Controller $\rightarrow$ Service $\rightarrow$ Repository $\rightarrow$ Entity), mientras que el patrón *Cache-Aside* con @Cacheable y @CacheEvict en Redis redujo las consultas repetitivas a la base de datos en un 80%, optimizando drásticamente la latencia ante accesos concurrentes.

Conclusión

En conclusión, la integración de Angular 21 con un backend en Spring Boot demuestra que es posible construir una plataforma de gestión deportiva altamente escalable y mantenible. El uso de cookies *HttpOnly* junto con Redis resuelve eficientemente la revocación de sesiones *stateless*, elevando los estándares de seguridad sin sacrificar la velocidad de respuesta. Asimismo, la estrategia de caché para la lectura de datos recurrentes, combinada con una base de datos poblada de forma controlada mediante Flyway, confirma que la arquitectura propuesta está técnicamente preparada para soportar una alta demanda transaccional, garantizando estabilidad y un rendimiento óptimo en producción.

EQUIPO D

ESCUDERO PLAZA MARIA DEL ROSARIO
 LOOR MEDRANDA MARLON TAYLOR
 CASTRO ESPINOZA KEVIN MOISES

Resultados obtenidos

.....
.....

Conclusión

```
.. aaaaaaaaaaaaaaaaaaaaaaa . aaaaaaaaaaaaaaaaaaaaaaa . aaaaaaaaaaaaaaaaaaaaaaa . aaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaa . aaaaaaaaaaaaaaaaaaaaaaa . aaaaaaaaaaaaaaaaaaaaaaa
```

EQUIPO E

RIVERA SUAREZ JONATHAN JAVIER
ALAVA ALVARADO JEAN PIERRE
TEJADA BAJAÑA LUIS ALEJANDRO

Resultados obtenidos

e implementó exitosamente la capa de integración del PFC bajo arquitectura en capas (Controller-Service-Repository-Entity) con Spring Boot 3.3, PostgreSQL 16 y Redis 7. Se logró autenticación stateless mediante JWT almacenado en cookie HttpOnly, con blacklist de JTI en Redis que invalida efectivamente el token al hacer logout, verificado mediante inspección directa con `redis-cli`. El CRUD de vehículos quedó operativo con paginación (Pageable), filtros combinables y ordenamiento dinámico, respaldado por 60 registros semilla cargados vía Flyway. El patrón cache-aside (@Cacheable/@CacheEvict) redujo significativamente el tiempo de respuesta en consultas repetidas, evidenciado en las pruebas unitarias (repositorio, servicio, JWT y blacklist) y en el benchmark de Speedup ejecutado sobre el listado principal.

Conclusión

La implementación demostró que la gestión de estado stateless con JWT y blacklist en Redis resuelve el problema de invalidación de sesión sin sacrificar la escalabilidad horizontal, evitando el uso de sticky sessions. La arquitectura en capas facilitó la separación de responsabilidades y la mantenibilidad del código, mientras que el patrón cache-aside evidenció cuantitativamente su impacto positivo en el rendimiento, justificando la complejidad operacional adicional que implica Redis frente al costo-beneficio observado. Las pruebas unitarias con H2 y Mockito garantizaron la validez de las reglas de negocio de forma aislada. Se concluye que el stack Java 21/Spring Boot/PostgreSQL/Redis es adecuado para sistemas web que requieren rendimiento y consistencia de datos.

EQUIPO F

CRUZ PEREZ JUSTYN KEITH
BELTRAN MONTIEL FRED ADRIAN

PALLO PINTO DANIEL ALEJANDRO

Resultados obtenidos

Objetivo General

Diseñar e implementar la capa de integración del Proyecto Fin de Curso, conectando el frontend con el backend a través de una arquitectura documentada con Modelo C4, implementando acceso a datos con ORM y patrón Repository, caché con Redis (patrón cache-aside), y midiendo el impacto cuantitativo de la caché en el rendimiento de las consultas, con cobertura de pruebas unitarias $\geq 70\%$.

Objetivos Específicos:

- OE1. **Documentar** la arquitectura completa del PFC con el Modelo C4 en los niveles de Contexto, Contenedores y Componentes, incluyendo mínimo 3 ADR que justifiquen las decisiones arquitectónicas más relevantes.
- OE2. **Implementar** el modelo de datos completo del PFC con ORM (Doctrine, Entity Framework Core o Hibernate/JPA), incluyendo migraciones versionadas, seeders con ≥ 50 registros realistas, y CRUD con paginación, ordenamiento dinámico y búsqueda con filtros múltiples.
- OE3. **Implementar** el patrón de caché cache-aside con Redis, midiendo el speedup de la caché en la consulta principal del PFC con la fórmula $S = T_{sin}/T_{con}$ en 10 repeticiones.
- OE4. **Desarrollar** pruebas unitarias para la capa Repository con cobertura $\geq 70\%$, generando el reporte de cobertura e incluyéndolo en el informe.

Conclusión

Objetivo General

Diseñar e implementar la capa de integración del Proyecto Fin de Curso, conectando el frontend con el backend a través de una arquitectura documentada con Modelo C4, implementando acceso a datos con ORM y patrón Repository, caché con Redis (patrón cache-aside), y midiendo el impacto cuantitativo de la caché en el rendimiento de las consultas, con cobertura de pruebas unitarias $\geq 70\%$.

Objetivos Específicos:

- OE1. **Documentar** la arquitectura completa del PFC con el Modelo C4 en los niveles de Contexto, Contenedores y Componentes, incluyendo mínimo 3 ADR que justifiquen las decisiones arquitectónicas más relevantes.
- OE2. **Implementar** el modelo de datos completo del PFC con ORM (Doctrine, Entity Framework Core o Hibernate/JPA), incluyendo migraciones versionadas, seeders con ≥ 50 registros realistas, y CRUD con paginación, ordenamiento dinámico y búsqueda con filtros múltiples.
- OE3. **Implementar** el patrón de caché cache-aside con Redis, midiendo el speedup de la caché en la consulta principal del PFC con la fórmula $S = T_{sin}/T_{con}$ en 10 repeticiones.
- OE4. **Desarrollar** pruebas unitarias para la capa Repository con cobertura $\geq 70\%$, generando el reporte de cobertura e incluyéndolo en el informe.

EQUIPO G

FIGUEROA MORALES BRYAN JAVIER

VELEZ LOPEZ RICARDO ELIAS

ROMERO MENDEZ BRYAN STEVEN

Resultados obtenidos

El objeto Request encapsula la información que el cliente envía al servidor dentro de una petición HTTP, incluyendo parámetros, cabeceras, cookies y el cuerpo del mensaje. En plataformas basadas en Java, el contenedor web transforma cada petición entrante en un objeto que es entregado al componente encargado de procesarla. Este objeto constituye el punto de entrada de datos hacia la lógica de negocio y es la base sobre la cual se construyen mecanismos de enrutamiento, validación y control de acceso. El objeto Response representa la información que el servidor devuelve al cliente tras procesar la solicitud; incluye el cuerpo del contenido, el código de estado HTTP y las cabeceras de salida, entre ellas las utilizadas para el envío de cookies mediante métodos. Este objeto es responsable de convertir el resultado del procesamiento interno del servidor en un mensaje HTTP comprensible para el navegador, cerrando así el ciclo petición-respuesta que caracteriza a la arquitectura cliente-servidor.

Conclusión

El objeto Application almacena información compartida por todas las sesiones activas dentro de un mismo directorio virtual o contexto de despliegue, sirviendo como espacio de datos común a nivel de toda la aplicación. Por su parte, el objeto Page (o el

ámbito de página) delimita el ciclo de vida más corto dentro del modelo de objetos integrados, existiendo únicamente durante el procesamiento de una única petición-respuesta. La combinación jerárquica de estos ámbitos petición, sesión y aplicación define el modelo clásico de gestión de estado en aplicaciones web dinámicas, y su diseño influye directamente en decisiones posteriores de escalabilidad, como el uso de componentes sin estado para facilitar el despliegue en granjas de servidores.

EQUIPO H

FAJARDO MONTES MICHAEL XAVIER

CARVAJAL LOOR JOHAN STALIN

MARISCAL CABRERA JAIME JOSUE

Resultados obtenidos

Durante el desarrollo de esta práctica se logró implementar y verificar correctamente el sistema web BIOPET, integrando un frontend desarrollado en Angular con un backend en Spring Boot y una base de datos PostgreSQL ejecutada mediante Docker. Se comprobó el funcionamiento del inicio de sesión con JWT almacenado en una cookie HttpOnly, el cierre de sesión con invalidación del token mediante Redis y el acceso protegido a los endpoints. También se ejecutó el CRUD completo de mascotas, incluyendo creación, consulta, actualización, eliminación lógica, paginación, ordenamiento y filtros por nombre, especie y raza. Flyway permitió crear el esquema y cargar los datos semilla automáticamente. Además, se validó el uso de caché con Redis, su TTL y la invalidación después de modificar registros. Las 53 pruebas automatizadas finalizaron correctamente, la cobertura JaCoCo superó el 70 % requerido y el benchmark evidenció una reducción en el tiempo promedio de respuesta al utilizar caché.

Conclusión

La práctica permitió comprender de forma aplicada cómo se integran distintas tecnologías en una aplicación web completa y funcional. El uso de Spring Boot, Angular, PostgreSQL, Redis, Flyway y Docker facilitó la construcción de una solución organizada, segura y reproducible. También se comprobó la importancia de implementar pruebas automatizadas, control de versiones, migraciones de base de datos y mecanismos de caché para mejorar la calidad y el rendimiento del sistema. La autenticación mediante JWT en cookie HttpOnly y la blacklist almacenada en Redis fortalecieron la seguridad del proyecto. En conclusión, BIOPET cumplió con los requisitos técnicos planteados y permitió consolidar conocimientos sobre arquitectura por capas, ORM, persistencia, caché, pruebas, integración continua y documentación de software.

EQUIPO I

RIOS CUYABAZO JHON KEVIN

ARIAS MOREIRA MAYBELIN GREGORIA

CASTRO PALMA JUSTYN MOISES

Resultados obtenidos

Lo que se obtuvo como resultado de este trabajo fue el logro de la contenedorización completa del entorno de desarrollo utilizando Docker, integrando servicios independientes para la aplicación Laravel, MySQL y Redis. La implementación del patrón Cache-Aside en la capa Repository optimizó drásticamente el rendimiento de las consultas paginadas con filtros dinámicos, alcanzando un Speedup cuantitativo promedio superior a 2.5x en el benchmark de diez iteraciones consecutivas. Adicionalmente, se alcanzó una cobertura de pruebas unitarias del 72% para la capa de persistencia de datos. Finalmente, la arquitectura del sistema quedó documentada formalmente mediante el Modelo C4 y tres Registros de Decisión Arquitectónica (ADR) validados.

Conclusión

Se concluye que el uso estratégico de Redis como memoria caché centralizada reduce significativamente la carga de lectura en bases de datos relacionales, siendo una solución indispensable para garantizar la escalabilidad horizontal bajo alta concurrencia. Asimismo, el Modelo C4 y los documentos ADR demostraron ser herramientas metodológicas superiores a UML tradicional, facilitando una gobernanza arquitectónica clara y alineada a las restricciones de carga cognitiva del equipo. Por último, la correcta abstracción de datos mediante el patrón Repository no solo mitiga el acoplamiento técnico, sino que viabiliza pruebas unitarias automatizadas que garantizan la estabilidad del software ante cambios futuros.

GUERRERO ULLOA GLEISTON CICERON
CI: 0913531752
DOCENTE RESPONSABLE

PONCE ORDOÑEZ JESSICA ALEXANDRA
CI: 1205316456
COORDINADOR DE CARRERA

